2.4 基本运算方法

补码（+、−、 ×、 ÷ ）➡ 定点数运算

原码（+、 −、 ×、 ÷）
移码（+、 −）
} 浮点数运算

2.4.1 定点数的运算

定点数一般用补码表示；

符号位参加运算。

重点：基于补码的加、减、乘、除法

回顾

求变补（变补，机器负数），即已知$[X]_{补}$，求$[-X]_{补}$

$[X]_{补}$的代码连同符号位一起变反，末位再加1，即得到$[-X]_{补}$

[例]已知$[X]_{补}$=01010110，求$[-X]_{补}$

解：$[X]_{补}$=0 1010110　　不区分正负数

↓↓↓↓↓↓↓↓　　连同符号位一起变反

10101001

\+ 1　　末位加1

$[-X]_{补}$=10101010

1. 补码的加减法

$$(X+Y)_{补}=X_{补}+Y_{补} \quad (1)$$

$$(X-Y)_{补}=X_{补}+(-Y)_{补} \quad (2)$$

数学依据：

$$(X+Y)_{补}=X+Y+2^n=X+Y+2^n+2^n=X_{补}+Y_{补}$$

$$(X-Y)_{补}=X-Y+2^n=X-Y+2^n+2^n=X_{补}+(-Y)_{补}$$

全过程以2^n为模，即除以2^n后取余数。

其中，$(-Y)_{补}=[Y_{补}]_{变补}$

$(-Y)_{补}$：Y的符号置反后再表示成补码

$[Y_{补}]_{变补}$：$Y_{补}$连同符号一起变反、末尾+1

【例】求$(X+Y)_{补}$

1) X=3　$X_{补}$=0 0011

Y=2　$Y_{补}$=0 0010 ＋

0 0101（+5补码）

2) X= –3 $X_{补}$=1 1101

Y= –2 $Y_{补}$=1 1110 ＋

1 1011（-5补码）

【例】求$(X-Y)_{补}$

1) X= 4　$X_{补}$=0 0100

Y= –5　$Y_{补}$=1 1011

$(-Y)_{补}$=0 0101 ＋

0 1001（+9补码）

2) X= –4　$X_{补}$=1 1100

Y= 5　$Y_{补}$=0 0101

$(-Y)_{补}$=1 1011 ＋

1 0111（–9补码）

※补码表示与变补运算的区别

【举例】

$1\underline{0}101_{原} \xrightarrow{补码表示} 1\underline{1}011$

$0\underline{0}101_{原} \xrightarrow{补码表示} 0\underline{0}101$

符号位不变

负数尾数改变，正数尾数不变。

$1\underline{0}011_{补} \xrightarrow{变补运算} 0\underline{1}101$

$0\underline{0}011_{补} \xrightarrow{变补运算} 1\underline{1}101$

符号位变反，

尾数变反、末尾加1

$(-Y)_{补}$也称为$Y_{补}$的机器负数。

例1：X=0.1011　　Y=-0.0101，求X+Y=?

解：$[X]_{补}$=0.1011，$[Y]_{补}$=1.1011

$$
\begin{array}{rl}
 & [X]_{补}=0.\ 1\ 0\ 1\ 1 \\
+ & [Y]_{补}=1.\ 1\ 0\ 1\ 1 \\
\hline
 & 10.\ 0\ 1\ 1\ 0
\end{array}
$$

模数丢掉

$[X]_{补}+[Y]_{补}=0.0110$

$[X+Y]_{补}=0.0110$

$X+Y=0.0110$

例1：X=-0.1010　Y=-0.0100，求X+Y=?

解：$[X]_{补}$=1.0110，$[Y]_{补}$=1.1100

$$
\begin{array}{rl}
 & [X]_{补}=1.\,0\,1\,1\,0 \\
+ & [Y]_{补}=1.\,1\,1\,0\,0 \\
\hline
 & 11.\,0\,0\,1\,0
\end{array}
$$

模数丢掉

$[X]_{补}+[Y]_{补}=1.0010$

$[X+Y]_{补}=1.0010$

$X+Y=-0.1110$

例：X=0.1001　　Y=0.0110，求$[X-Y]_{补}$=?

解：$[X]_{补}$=0.1001，$[Y]_{补}$=0.0110，　$[-Y]_{补}$=1.1010

$$
\begin{array}{rr}
 & [X]_{补}=0.\ 1\ 0\ 0\ 1 \\
+ & [-Y]_{补}=1.\ 1\ 0\ 1\ 0 \\
\hline
 & 10.\ 0\ 0\ 1\ 1
\end{array}
$$

$[X-Y]_{补}=[X]_{补}+[-Y]_{补}=0.0011$

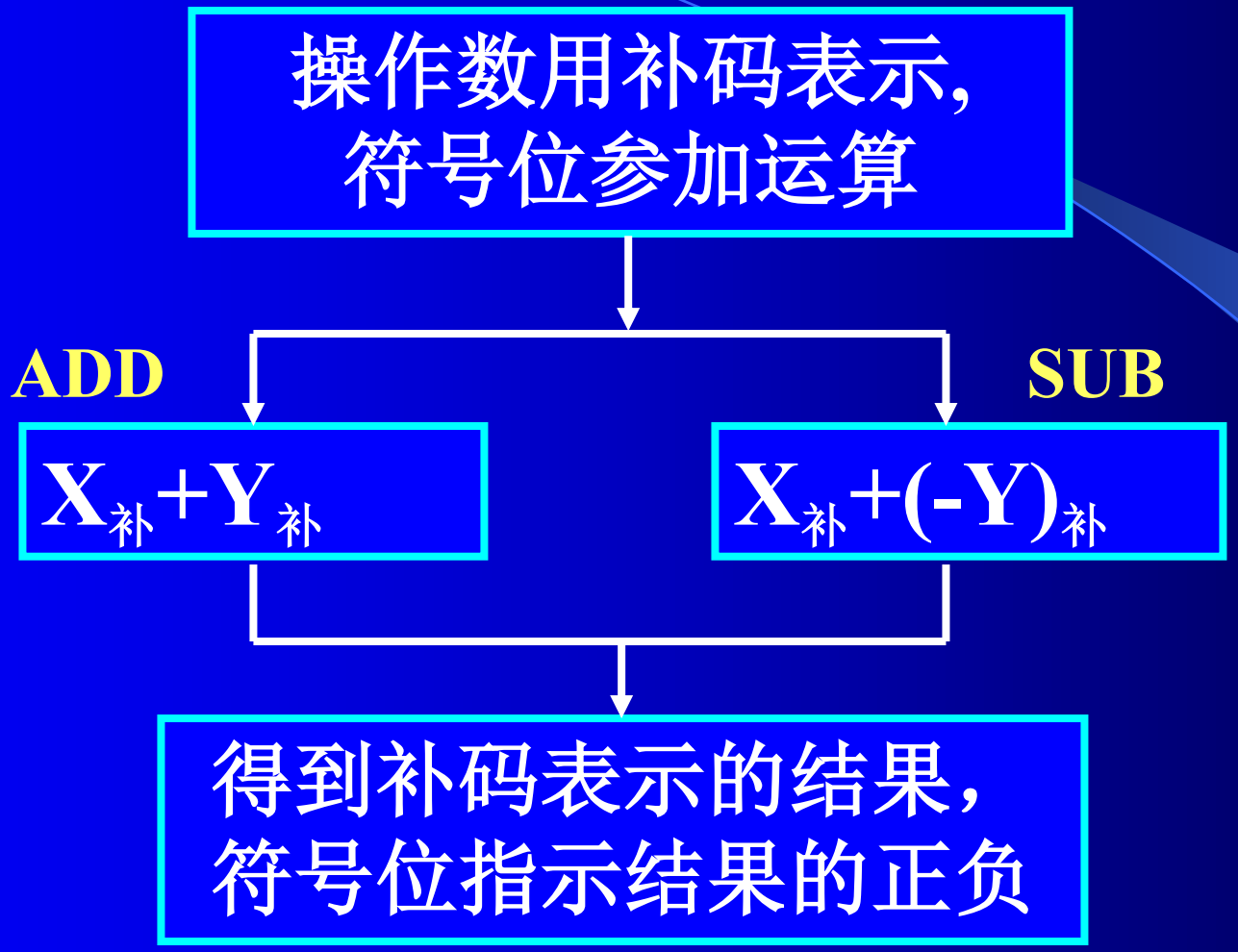
说明：

（1） 两个数不管正负，均用补码表示，符号位应当做数值参加运算，符号位相加产生的进位要丢掉，结果为补码。

（2）用补码表示的两个数进行加法运算，结果为补码。

（3）符号位与数值位一样参加运算。

（2）补码加减运算流程



（3）逻辑实现

#控制信号

加法器输入端：

Sub：控制MUX和Cin

A：输入$A_{补}$

B：输入$B_{补}$

加法器输出端：

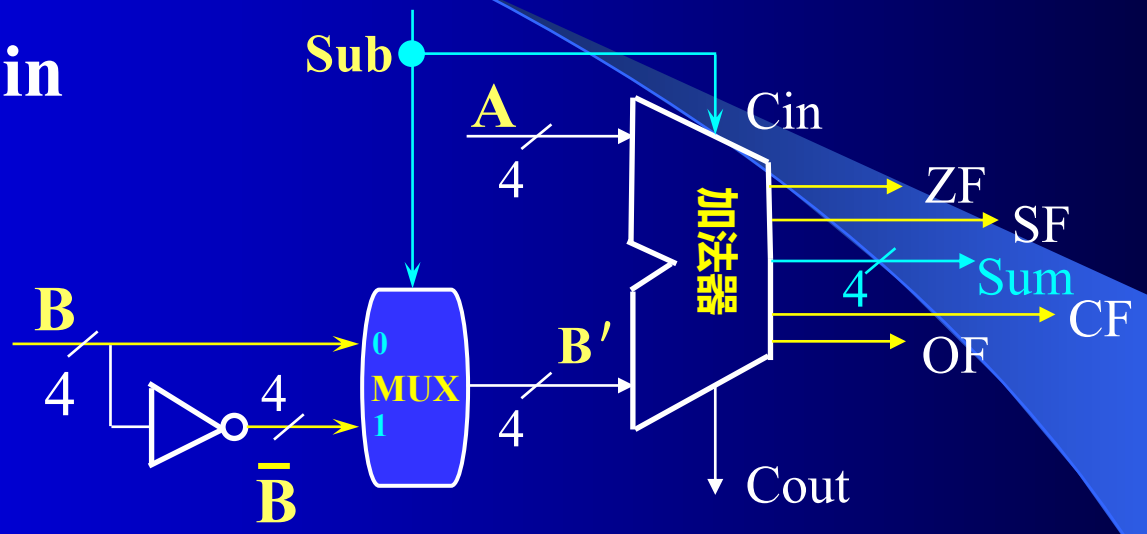
Sum: 加法结果

Cout:进位信号

ZF: 0标志位；**SF**：符号标志；**CF**:进位/借位标志；

OF:溢出标志。

#补码加减运算器框图



补码加/减运算部件逻辑

小结



问题： 补码加减运算的用途是什么？

用于实现带符号整数的加减运算端。

- 补码加减运算要点和运算部件
  - 加、减法运算统一采用加法来处理
  - 符号位和数值位一起参与运算
  - 直接用ALU实现两个数的加运算

补码加法的几种情况-溢出引入

正常结果

```
  0 .1 0 1 0
+ 0 .0 1 0 0 0
-------------
  0 .1 1 1 0 1
```

```
  1 .1 0 1 0 1
+ 1 .1 1 0 0 0
-------------
1 1 .0 1 1 0 1
```

模数舍去

正常结果

正正得负

正溢出

```
  0 .1 0 1 0 1
+ 0 .1 1 0 0 0
-------------
  1 .0 1 1 0 1
```

负负得正

负溢出

```
  1 .0 0 1 0 1
+ 1 .1 1 0 0 0
-------------
1 0 .1 1 1 0 1
```

计算机如何识别运算结果是否溢出？

溢出及其判别方法

1. 溢出

是指运算结果超出机器所能表示的数值范围的现象。 当发生溢出时，计算结果是错误的，计算机必须及时作出相应处理。 --报错

（4）溢出判断

*溢出的判断规则

思考：*在什么情况下可能产生溢出？*

溢出及其判别方法

2. 溢出产生的条件



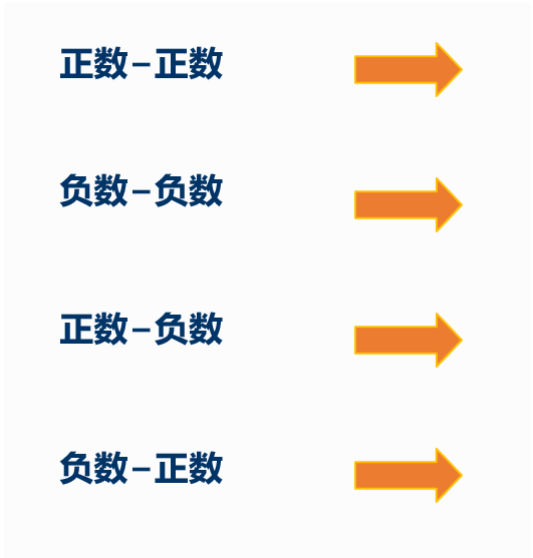
溢出及其判别方法

2. 溢出产生的条件

正数+正数	→	可能溢出
负数+负数	→	可能溢出
正数+负数	→	不可能溢出
负数+正数	→	不可能溢出

溢出及其判别方法

2. 溢出产生的条件



溢出及其判别方法

2. 溢出产生的条件

正数－正数	→	不可能溢出
负数－负数	→	不可能溢出
正数－负数	→	可能溢出
负数－正数	→	可能溢出

结论： 两异号数相加或两同号数相减一定不会产生溢出，
两同号数相加或两异号数相减时才有可能产生溢出。

（4）溢出判断

［假设］补码表示的A、B两数做加减运算

A：4位尾数，1位符号(S_A)
B：4位尾数，1位符号(S_B)

> 补码，故符号位也参加运算

结果的符号，记为S_f

符号位的进位，记为C_f

尾数最高位的进位，记为C

1）A=3、B=2

3+2：　0 0011

　　0 0010 ＋

　　0 0101 正确

2）A=10、B=7

10+7：　0 1010

　　0 0111 ＋

　　1 0001 正溢

3）A= -3、B= -2

-3+(-2)：1 1101

　　1 1110 ＋

　　1 1011 正确

4）A= -10、B= -7

-10+(-7)：1 0110

　　1 1001 ＋

　　0 1111 负溢

5）A=6、B= -4

6+(-4)：0 0110

　　1 1100 ＋

　　0 0010 正确

6）A= -6、B=4

-6+4：　1 1010

　　0 0100 ＋

　　1 1110 正确

①硬件判断逻辑一（根据S_A、S_B与S_f的关系）

（2）A=10 B=7

10+7：0 1010

0 0111 +

正溢 1 0001

（4）A= -10 B= -7

-10+(-7)：1 0110

1 1001+

负溢 0 1111

溢出逻辑= $\overline{S}_A\overline{S}_BS_f + S_AS_B\overline{S}_f$

- 溢出逻辑： **正正得负 负负得正**

练习

- x=-0.100 y =-0.101,求x+y=

练习

- x=-0.100 y =-0.101,求x+y=

解法： [x]补=1.100 [y]补=1.011

[x+y]补 = 0.111

$$Overflow = \bar{f}_0 \bar{f}_1 f_s + f_0 f_1 \bar{f}_s$$
$$= 0 \cdot 0 \cdot 0 + 1 \cdot 1 \cdot 1 = 1$$

两个负数相加，结果为正，溢出

②硬件判断逻辑二（根据C_f与C的关系）

1）A=3　B=2

3+2：　0 0011

C_f=0　0 0010 ＋

C =0　0 0101 正确

2）A=10　B=7

10+7：　0 1010

C_f=0　0 1 0111 ＋

C =1　1 0001 正溢

3）A= -3　B= -2

-3+(-2)：　1 1101

C_f=1　1 1 1110 ＋

C =1　1 1011 正确

4）A= -10　B= -7

-10+(-7)：　1 0110

C_f=1　1 1 1001 ＋

C =0　0 1111 负溢

5）A=6　B= -4

6+(-4)：　0 0110

C_f=1　1 1 1 1100 ＋

C =1　0 0010 正确

6）A= -6　B=4

-6+4：　1 1010

C_f=0　0 0100 ＋

C =0　1 1110 正确

①硬件判断逻辑一（S_A、S_B与S_f的关系）

溢出逻辑= $\overline{S}_A\overline{S}_B S_f + S_A S_B \overline{S}_f$

②硬件判断逻辑二（C_f与C的关系）

溢出逻辑= $C_f \oplus C$

若最高数值位产生向符号的进位而符号位不产生进位时，发生正溢出。
若最高数值位无进位 而符号位有进位时，发生负溢出 。

③硬件判断逻辑三（从双符号位）

练习

- x=-0.100 y =-0.101,求x+y=

练习

- x=-0.100 y =-0.101,求x+y=

解法： [x]补=1.100 [y]补=1.011

[x+y]补 = 0.111

$$Overflow = C_f \oplus C_n = 1+0=1$$

两个负数相加，结果为正，溢出

1）3+2：

00 0011

00 0010 ＋

00 0101 正确

2）10+7：

00 1010

00 0111 ＋

01 0001 正溢

第 1 符号位 S_{f1}

3）-3+(-2)：

11 1101

11 1110 ＋

11 1011 正确

4）-10+(-7)：

11 0110

11 1001 ＋

10 1111 负溢

第 2 符号位 S_{f2}

5）6+(-4)：

00 0110

11 1100 ＋

00 0010 正确

6）-6+4：

11 1010

00 0100 ＋

11 1110 正确

①硬件判断逻辑一（S_A、S_B与S_f的关系）

溢出逻辑= $\overline{S}_A\overline{S}_BS_f + S_AS_B\overline{S}_f$　1溢出，0无溢出

②硬件判断逻辑二（C_f与C的关系）

溢出逻辑= $C_f \oplus C$　1溢出，0无溢出

③硬件判断逻辑三（双符号位S_{f1}、S_{f2}）

溢出逻辑= $S_{f1} \oplus S_{f2}$

00/11-正确；01-正溢；10-负溢；

练习

- x=-0.100 y =-0.101,求x+y=

练习

- x=-0.100 y =-0.101,求x+y=

解法：[x]补=11.100 [y]补=11.011

[x+y]补 = 10.111

$$\boldsymbol{Overflow} = S_{f1} \oplus S_{f2} = \mathbf{1}$$

两个负数相加，结果为正，溢出

2、原码加减运算

[符号位单独处理、数值位加减]

先比较两数符号：

求和	求差
$3_{原}+2_{原}$	$3_{原}+[-2]_{原}$
$3_{原}-[-2]_{原}$	$3_{原}-2_{原}$

①加法：同号数值位求和，异号求差；

②减法：异号数值位求和，同号求差；

※求和时：数值位相加，和的符号取被加数（被减数）符号

[若最高位产生进位，则结果有溢出。

※求差时：被加数（被减数）与加数（减数）求补后相加。

⊙最高数值位有进位，相加结果为正，数值位正确；符号取被加数（被减数）的符号。

⊙最高数值位无进位，相加结果为负，得到数值位的补码，需对结果求补还原为绝对值形式的数值位；符号位与被加数（被减数）的符号相反。

[例] 已知 $[X]_{原} = 1.0011$，$[Y]_{原} = 1.1010$，计算$[X+Y]_{原}$

解：由原码加减运算规则知：同号相加，则求和，和的符号同被加数符号。

和的数值位为：0011 + 1010 = 1101

和的符号位为：1

$$[X+Y]_{原} = 1.1101$$

[例] 已知 $[X]_{原} = 1.0011$，$[Y]_{原} = 1.1010$，要求计算$[X-Y]_{原}$

解：由原码加减运算规则知：同号相减，则数值位求差

差的数值位为：$0011+(1010)_{求补} = 0011 + 0110 = 1001$

最高数值位无进位，表明加法结果为负，需对1001求补，还原为绝对值形式的数值位。即：$(1001)_{求补} = 0111$

差的符号位为$[X]_{原}$的符号位取反，即

$$[X-Y]_{原} = 0.0111$$

3、标准移码的加减

[符号位和数值部分一起处理]

$$[E1]_{移}+[E2]_{移}=2^{n-1}+E1+2^{n-1}+E2=2^n+E1+E2$$

$$=[E1+E2]_{补}\ （\mathrm{mod}\ 2^n）$$

$$[E1]_{移}-[E2]_{移}=2^{n-1}+E1+2^n-[E2]_{移}\ (\mathrm{mod}\ 2^n)$$

$$=2^{n-1}+E1+2^n-2^{n-1}-E2$$

$$=2^n+E1-E2=[E1-E2]_{补}\ （\mathrm{mod}\ 2^n）$$

[重要结论]

两数移码的加减等于两数加减后表示成的补码。

※补码和移码：符号位相反、数值位相同

因此，可得到移码的下列加减法则：

① 加法：直接将$[E1]_{移}$和$[E2]_{移}$进行模2^n加，结果的符号取反。

② 减法：先将减数$[E2]_{移}$求补，然后再与被减数 $[E1]_{移}$进行模2^n加，结果的符号取反。

[溢出判断]

进行模2^n相加时，如果两个加数与和数符号全相同，则发生了溢出。

[例]用4位移码计算“–7+（– 6）”和“–3 + 6”的值。

$[-7]_{移} = 0001$　　$[-6]_{移} = 0010$

$[-3]_{移} = 0101$　　$[6]_{移} = 1110$

$[-7]_{移} + [-6]_{移} = \underline{0}001 + \underline{0}010 = \underline{0}011_{补}$

（符号都为0，有溢出）

$[-3]_{移} + [6]_{移} = \underline{0}101 + \underline{1}110 = \underline{0}011_{补}$，

（符号取反后为 1011，其真值为+3）

思考：$[-7+(-6)]_{移}$=?　　$[-3+(6)]_{移}$ =?

[例]用四位移码计算“–7 –（– 6）”和“–3 – 5”

$[-7]_{移} = 0001$　　$[-6]_{移} = 0010$

$[-3]_{移} = 0101$　　$[5]_{移} = 1101$

$[-7]_{移} - [-6]_{移} = \underline{0}001 + \underline{1}110 = \underline{1}111_{补}$

符号取反后为 0111，其真值为–1。

$[-3]_{移} - [5]_{移} = \underline{0}101 + \underline{0}011 = \underline{1}000_{补}$

符号取反后为 0000，其真值为–8

2.4.2 定点数乘法

- √ 原码一位乘
- √ 补码一位乘
- √ 原码两位乘

1、原码一位乘

原码乘法 ⟶ 部分积累加、移位。

$X_{原}$　$Y_{原}$

↓　↓

[例] 0.1101×1.1011

乘积 $P = |X| \times |Y|$

符号 $S_P = S_X \oplus S_Y$

即乘积的符号位等于两数符号位的异或，乘积的数值部分等于 两数绝对值的乘积。

手工运算

$$
\begin{array}{rl}
 & 0.1101 \quad \leftarrow |X| \\
\times & 0.1011 \quad \leftarrow |Y| \\
\hline
 & 1101 \\
 & 1101 \\
 & 0000 \\
+ & 1101 \\
\hline
 & .10001111
\end{array}
$$

部分积

①加数只为|X|或0

添加符号：1.10001111

思考：1）加数的个数

起加）

2）加数的位数

解决办法：可将1次总

手工计算：

① 乘数的每一位对应一个部分积，当乘数的位为0，其部分积也是0，当乘数的位为1，其部分积是被乘数。

② 后面的部分积总要比它前面的部分积左移一个位置。

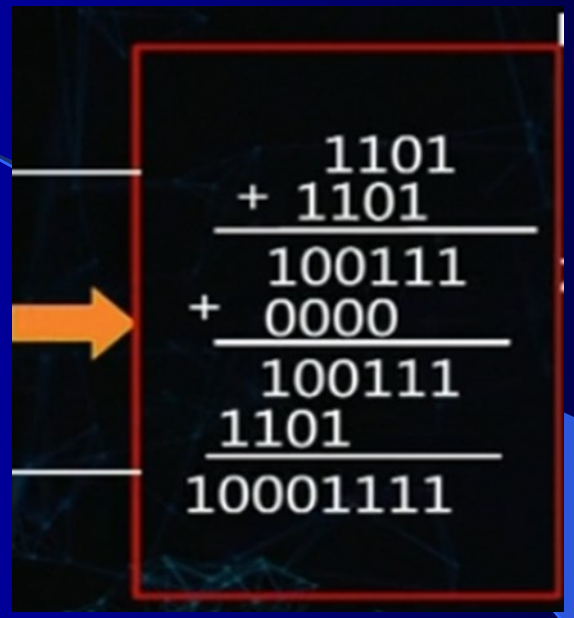
③ 多个部分积通过求和而得到的最后乘积。

④ 两个n位二进制整数的乘法可产生最大长度为2n位的积。

手工运算

$$
\begin{array}{rl}
 & 0.1101 \quad \leftarrow |X| \\
\times & 0.1011 \quad \leftarrow |Y| \\
\hline
 & 1101 \\
 & 1101 \\
 & 0000 \\
+ & 1101 \\
\hline
 & .10001111
\end{array}
$$

添加符号：1.10001111



问题：在加法器很难实现多个部分积同时相加

方法：每求得一个部分积，就与上次的部分积相加，得到的和，等待和下一次部分积相加，用累加的方法实现

手工运算

$$
\begin{array}{rl}
 & 0.1101 \quad \leftarrow |X| \\
\times & 0.1011 \quad \leftarrow |Y| \\
\hline
 & 1101 \\
 & 1101 \\
 & 0000 \\
+ & 1101 \\
\hline
 & .10001111
\end{array}
$$

添加符号：1.10001111



问题：人工计算时，相加数逐次向左偏移一位，最后乘积的位数是乘数位数的两倍。加法器的位数一般与寄存器位数相同，而不是寄存器位数的两倍。

方法：在求本次部分积时，前一次部分积的最低位不再参与运算了，因此可以将它右移一位，这样相加的部分积就可以用n位实现。

手工运算

$$
\begin{array}{rl}
 & 0.1101 \quad \leftarrow |X| \\
\times & 0.1011 \quad \leftarrow |Y| \\
\hline
 & 1101 \\
 & 1101 \\
 & 0000 \\
+ & 1101 \\
\hline
 & .10001111
\end{array}
$$

添加符号：1.10001111

方法：在部分积右移的时候，可以将乘数寄存器同时右移一位，就可以用乘数寄存器的最低一位来执行这种判断，而乘数寄存器的最高位可以空出来，用来接收部分积右移出来的一位。

问题：手工计算时，乘数的每一位是0还是1是直接可见的。而计算机内却看不见，怎么判断乘数的位值呢?

- 原码1位乘法

（1）算法原理[每次将1位乘数所对应的部分积与原部分积的累加和相加，并移位]

设置寄存器：

A：存放部分积累加和、乘积高位

B：存放被乘数

C：存放乘数、乘积低位

设置初值：

A = 00. 0000

B = |X| = 00. 1101

C = |Y| = . 1011

采用双符号位：因为累加时可能会有进位产生，非溢出

步数CR	条件	操作	A	右移1	C	C_n
			00.0000		1011	
0）	C_n=1	+B	+ 00.1101			
			00.1101		1011	
		→	00.0110	→	1.101	
1）	C_n=1	+B	+ 00.1101			
			01.0011		1101	
		→	00.1001	→	11.10	
2）	C_n=0	+0	+ 00.0000			
			00.1001		1110	
		→	00.0100	→	111.1	
3）	C_n=1	+B	+ 00.1101			
			01.0001		1111	
		→	00.1000	→	1111	

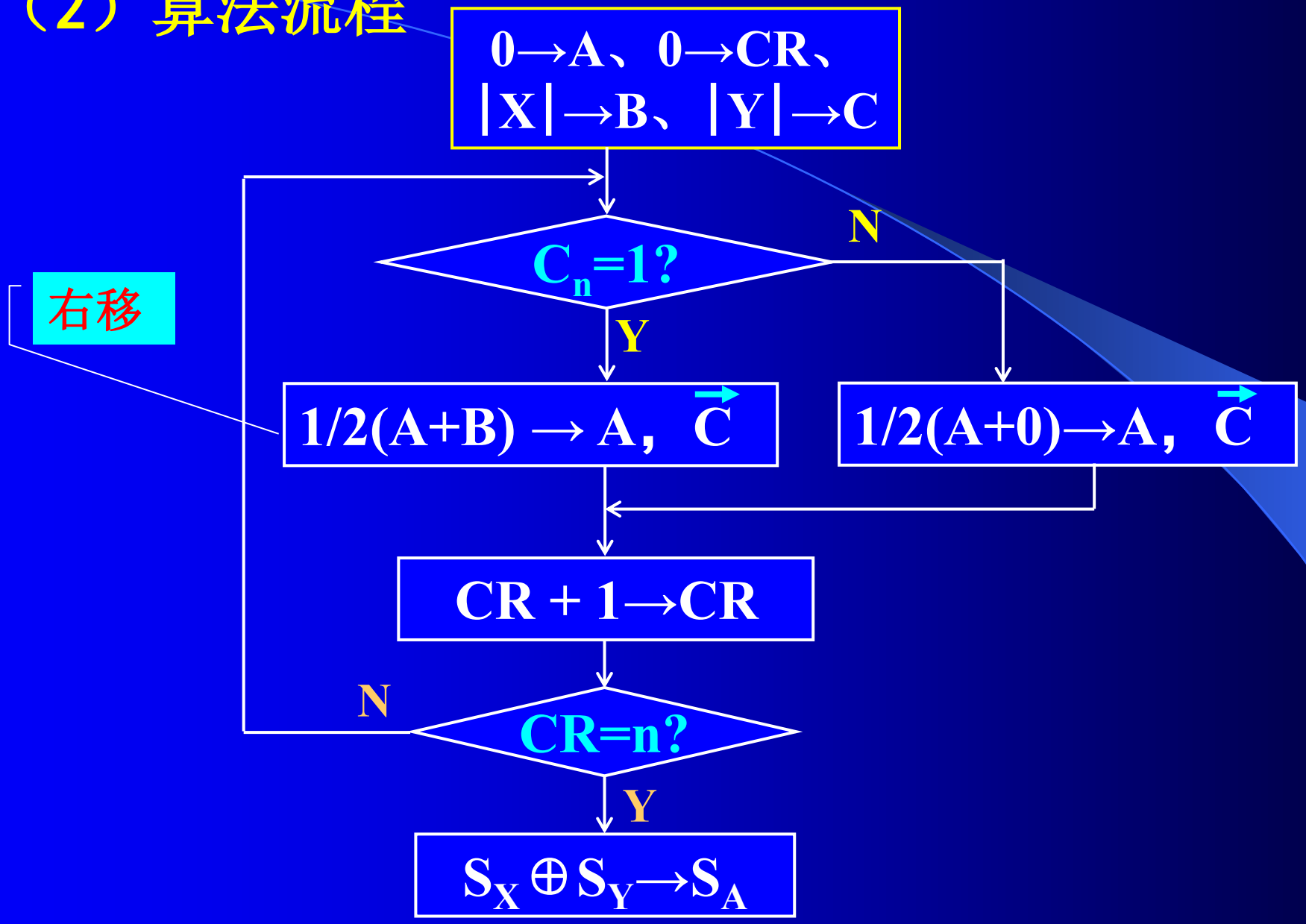
0.1011乘数

0.1101 —B
×0.1011 —C
1101
1101
0000
+1101
0.10001111

$X_{原} \times Y_{原} = 1.10001111$

A寄存器高位+C寄存器低位形成结果

（2）算法流程


0→A、0→CR、
|X|→B、|Y|→C
C_n=1?
N
Y
右移
1/2(A+B) → A，C
1/2(A+0)→A，C
CR + 1→CR
CR=n?
N
Y
$S_X \oplus S_Y \rightarrow S_A$

（3）运算规则

(1) 操作数和结果用原码表示；

(2) 绝对值运算，符号单独处理；

(3) 被乘数(B)、累加和(A)取双符号位；

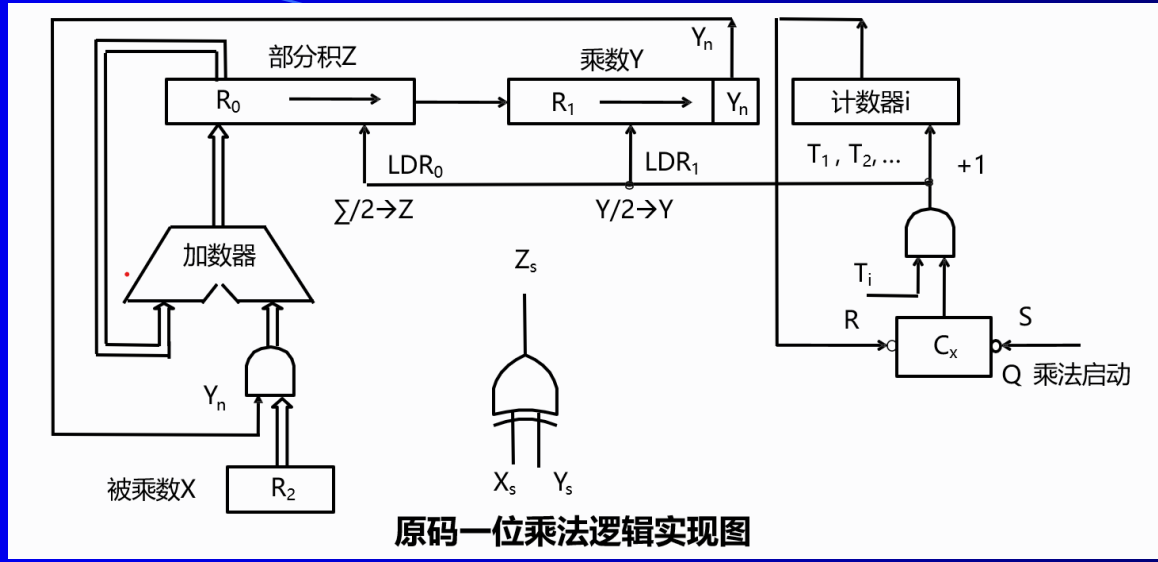
(4) 乘数末位C_n为判断位，其值决定下步操作；

(5) 若Cn=1，则部分积加B，并右移一位；

若Cn=0，则部分积加0，并右移一位；

(6) 作n次循环（累加、右移）。

（4）原码一位乘法逻辑实现



原码一位乘法逻辑实现图

1　乘法开始时，“启动”信号使控制触发器C_x置“1”，于是开启时序脉冲T。

2　当乘数寄存器R_1最末位为“1”时，部分积Z_i和被乘数X在加法器中相加，其结果输出至R_0的输入端。

3 一旦打入控制脉冲T到来，控制信号LDR_0使部分积右移一位，与此同时，R_1也在控制信号LDR_1作用下右移一位，且计数器i计数一次。

4 当计数器i=n时，计数器的溢出信号使触发器C_x置“0”，关闭时序脉冲 T，乘法宣告结束。

5 若将R_0和R_1连接起来，乘法结束时乘积的高n位部分在R_0，低n位部分在R_1，R_1中原来的乘数Y由于移位而全部丢失。

（5）原码一位乘特点

1 符号位需要单独运算，最后给运算结果以正确的符号；

2 对于采用补码存储的机器，从存储器中取出的是操作数的补码，需先将其转换成原码，最后还要将结果转换为补码存放到机器中，这样做很不方便，而且影响速度。

2. 补码一位乘法

※Booth（比较法）

$$[XY]_{补}=[An]_{补}+(Y_1-Y_0)\times[X]_{补}$$

运算规则

1　符号位参加运算，运算的数均以补码表示；

2　被乘数一般取双符号位参加运算；

3　乘数可取单符号位；

4　乘数末位增设附加位Y_{n+1}，且初值为0。

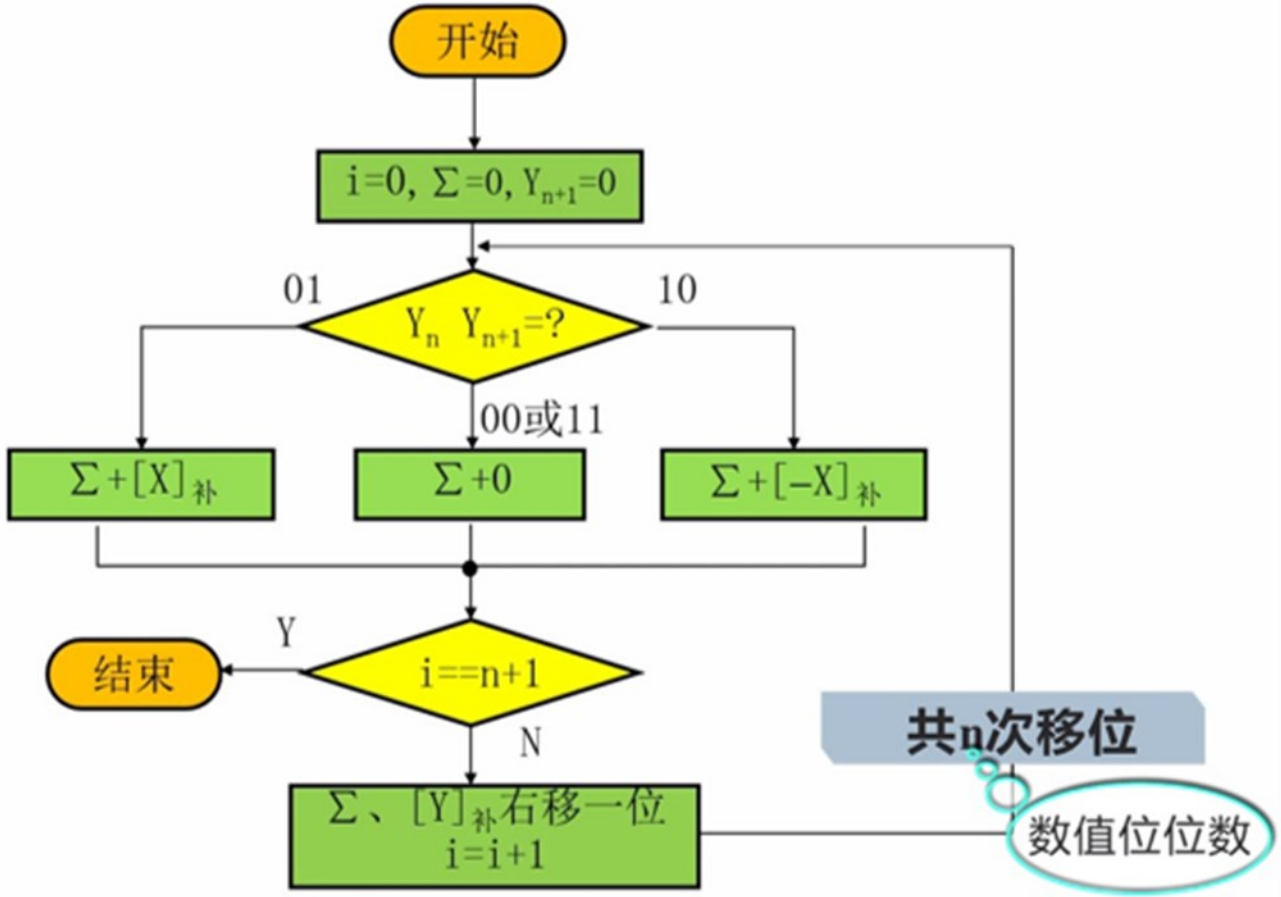
Y_n（高位）	Y_{n+1}（低位）	操作
0	0	部分积右移一位
0	1	部分积加$[X]_{补}$，右移一位
1	0	部分积加$[-X]_{补}$，右移一位
1	1	部分积右移一位

5　从低位到高位，依次判断相邻的两位乘数的状态，根据比较结果按右表操作。

6 按照上述算法进行n+1步累加操作，n步右移操作。

最后一步不移位，仅根据Y_0与Y_1的比较结果作相应的运算。

2 补码一位乘法流程图（booth一位乘法）



[例] X= -0.1101，Y= -0.1011，计算$[XY]_{补}$

初始化A=00.0000，$B=X_{补}$=11.0011，$-B=-X_{补}$=001101

$C=Y_{补}$=1.0101

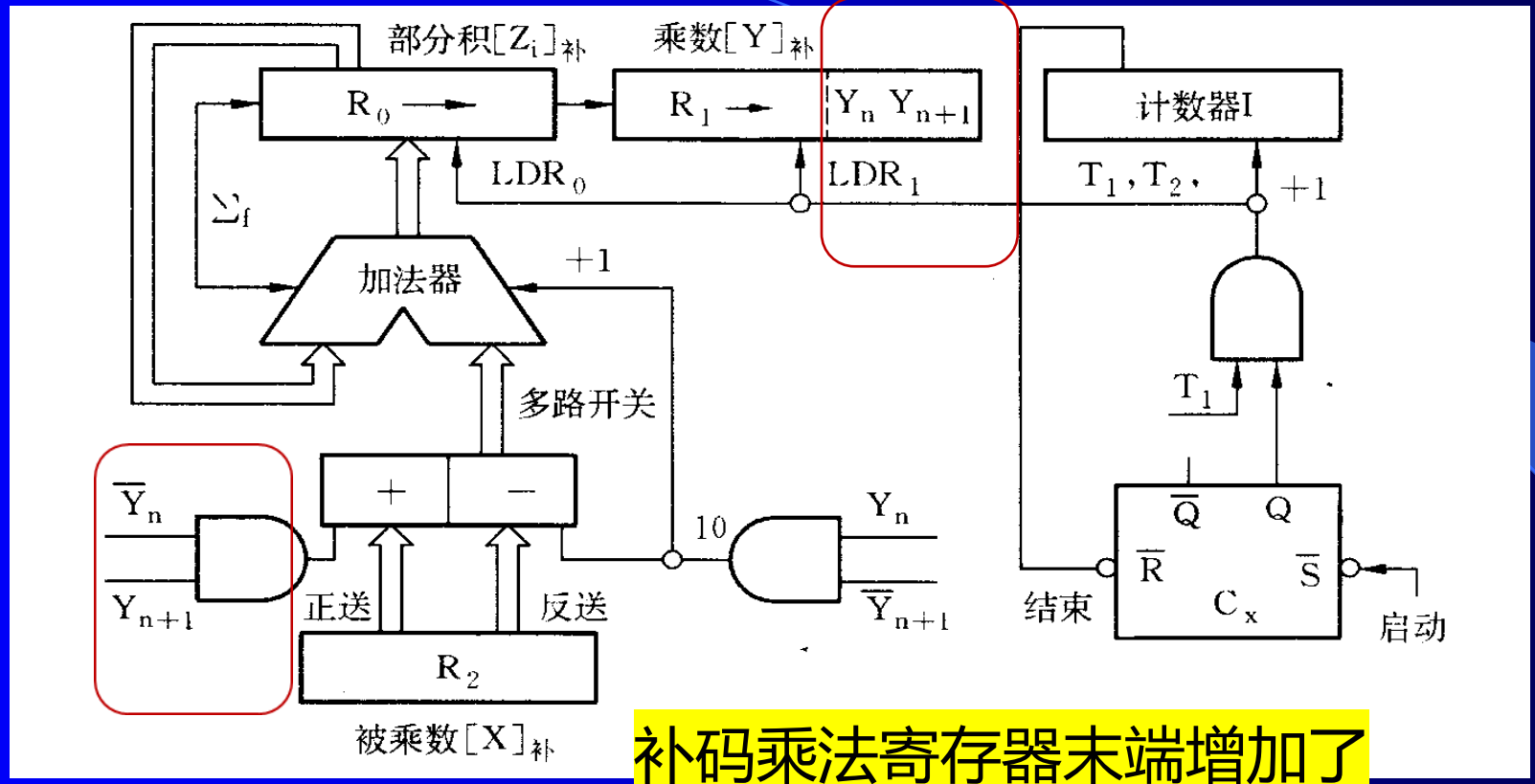
步数	C_nC_{n+1}条件	操作	A	C	C_{n+1}	
			000000	10101	0	
1	1 0	-B	+ 001101			
			001101	10101	0	右移1位→
			000110	11010	1	
2	0 1	+B	+ 110011			
			111001	11010	1	右移1位→
			111100	11101	0	

步数	C_nC_{n+1}条件	操作	111100 11101 0
3	1 0	-B	+ 001101
			1 001001 11101 0 右移1位→
			000100 11110 1
4	0 1	+B	+ 110011
			110111 11110 1 右移1位→
			111011 11111 0
校正	1 0	-B	+ 001101
			1 001000 11111 0 保持不变!!

$[XY]_{补}$= 001000 1111

= +0.1000 1111

（3） 补码一位乘法逻辑实现



补码乘法寄存器末端增加了附加位y_{n+1}，初值为0。

符号位参与运算，每次对y_n y_{n+1}进行判断后决定操作方式

Y_n (高位)	Y_{n+1} (低位)	操作
0	0	部分积右移一位
0	1	部分积加$[X]_{补}$，右移一位
1	0	部分积加$[-X]_{补}$，右移一位
1	1	部分积右移一位

※补码1位乘法除了比较法，还有校正法。

①乘数$Y_{补}$为正，乘以$X_{补}$累加，结果不校正；

②乘数$Y_{补}$为负，乘以$X_{补}$累加，结果$-X_{补}$校正；

（请参考教材）

多选题 1分

设置

关于原码一位乘法的下列描述中，正确的是（）

A 数据取绝对值参加运算

B 符号位单独处理

C 乘法执行过程中的所有移位都是算术移位

D 最后的结果由部分积寄存器和乘数寄存器共同保存

提交

多选题 1分

设置

关于补码一位乘法的下列描述中，正确的是（）

A 符号位和数值位一起参加运算

B 运算前需要自乘数寄存器Y后面补上Yn+1且初值为0

C 乘法执行过程中的所有移位都是算术移位

D 最后的结果由部分积寄存器和乘数寄存器共同保存

提交

多选题 1分

设置

3 计算机字长为n位，下列关于补码一位乘法操作过程中，正确的是（）

A 乘法过程中共执行n次加法和n-1部分积右移

B 乘法过程中共执行n-1次算术右移和n-1次加法运算

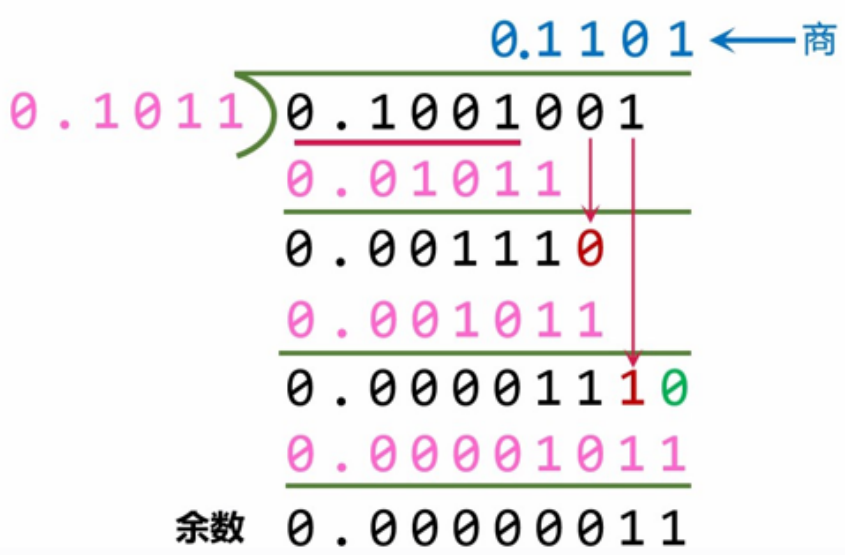
C 乘法过程中，部分积加0，[x]补还是[-x]补，取决于此时的YnYn+1

D 乘法过程中，右移部分积的目的是为了使部分积与下次的加数对齐。

提交

2.4.3 定点数的除法

除法手工运算



关键是判断是否够减？

- 不够减，商上0,
- 够减，商上1，求余数
- 除数右移一位
- 继续按规则上商直至所需位数

实现除法的关键： 比较余数、除数大小， 以决定上商。

2.3.4 定点除法运算

计算机要解决的问题：

判断是否够减？够减商1，不够减商0

如何判断够减？（先判后减和先减后判）

如何处理符号位？

如何提高运算速度？

1. 原码恢复余数除法



恢复余数法法则

被除数或余数减除数比较（$+[-Y]_{补}$）：

结果为正，商上1，左移一位

结果为负，商上0，加除数（$+[Y]_{补}$），左移一位

重复上述操作，直到商的位数符合要求为止

说明：　1）　计算机内的运算过程仍用补码。

2）在原码一位乘中通常有以下约定：

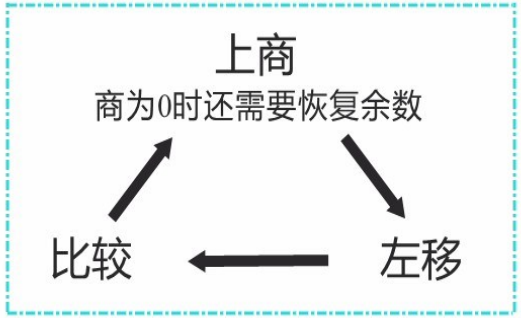
- 小数定点除法 |x| < |y|，整数定点除法 |x| > |y| （避免商溢出）
- 被除数不等于 0，除数不能为0

例 题



X=0.1001，Y=0.1011，求X，Y

- $[-Y]_{补}$ =1.0101
- Q=0.1101　　R= $0.0001*2^{-4}$



被除数/余数R	上商位Q_n	说明
00.1001		-Y
[-Y] + 11.0101		
11.1110	0	R<0
+ 00.1011		
00.1001		
← 01.0010		R=2R-Y
[-Y] + 11.0101		
00.0111	0.1	$R>0,Q_n=1$
← 00.1110		R=2R-Y
[-Y] + 11.0101		
00.0011	0.11	$R>0,Q_n=1$
← 00.0110		R=2R-Y
[-Y] + 11.0101		
11.1011	0.110	$R<0,Q_n=0,+Y$
+ 00.1011		
00.0110		
← 00.1100		R=2R-Y
[-Y] + 11.0101		
00.0001	0.1101	$R>0,Q_n=1$

恢复余数

恢复余数

恢复余数法的缺点

1某一次-Y的差值为负时,要多一次+Y恢复余数的操作,降低了执行度。

2由于部分余数的正、负是根据不同的操作数组合随机出现的， 恢复余数法会使得除法运算的实际操作次数不能固定，这会导致控制路比较复杂。

2. 原码不恢复余数除法



不恢复余数法法则

被除数或余数减除数比较（$+[-Y]_{补}$）：

结果为正，商上1，左移一位，减除数（$+[-Y]_{补}$）

结果为负，商上0，左移一位，加除数（$+[Y]_{补}$）

重复上述操作n+1次

原码一位除法

例 题



X=0.1001，Y=0.1011，求X／Y

- $[-Y]_{补}$ =1.0101
- Q=0.1101　　R= $0.0001*2^{-4}$

	被除数/余数R	上商位Q_n	说明
	00.1001		
[-Y] +	11.0101		
	11.1110	0	R<0,Q_n=0
	←11.1100		R=2R+Y
[Y] +	00.1011		
	00.0111	0.1	R>0,Q_n=1
	←00.1110		R=2R-Y
[-Y] +	11.0101		
	00.0011	0.11	R>0,Q_n=1
	←00.0110		R=2R-Y
[-Y] +	11.0101		
	11.1011	0.110	R<0,Q_n=0
	←11.0110		R=2R+Y
[Y] +	00.1011		
	00.0001	0.1101	R>0,Q_n=1



不恢复余数法法则

被除数或余数减除数比较（$+[-Y]_{补}$）：

结果为正，商上1，左移一位，减除数（$+[-Y]_{补}$）

结果为负，商上0，左移一位，加除数（$+[Y]_{补}$）

重复上述操作n+1次

3. 补码不恢复余数除法---自学

[算法思想]（$|X|<|Y|$）

被除数$X_{补}$、除数$Y_{补}$、余数r_i，i=0、1…

初始化：令$r_0=X_{补}$，比较r_0与$Y_{补}$符号，同号上商1，异号上商0

循环：i=1…n，按下表条件决定每步操作

r_i、$Y_{补}$数符	商	对应操作
同号	1	$r_{i+1}=2\times[r_i]_{补}-Y_{补}$
异号	0	$r_{i+1}=2\times[r_i]_{补}+Y_{补}$

$[-Y_{补}]\leftrightarrow+[Y_{补}]_{变补}$

$\times2\leftrightarrow$左移1位

商修正：符号位+1，末尾恒置1

余数修正：左移了n次，则余数=$2^{-n}\times r_n$

[例] X÷Y=+0.1000÷(−0.1010)=?

$R=X_{补}=\underline{00}1000$，$B=Y_{补}=\underline{11}0110$，$-B=001010$，$Q=00000$

步数	条件	操作	被除数	余数	商Q
1	$r_0 Y_{补}$异号	上商0	001000	r_0	00000
		$2r_0$ /←	010000		0000$\underline{0}$
		+B	+ 110110		
			1 000110	r_1	
2	$r_1 Y_{补}$异号	上商0	000110		00000
		$2r_1$ /←	001100		0000$\underline{0}$
		+B	+ 110110		
			1 000010	r_2	

$R=X_{补}=\underline{00}1000$，$B=Y_{补}=\underline{11}0110$，$-B=001010$

			000010 r_2	0000$\underline{0}$
3	$r_2Y_{补}$异号	上商0	000010	00000
		$2r_2$ /←	000100	0000$\underline{0}$
		+B	+ 110110	
			111010 r_3	
4	$r_3Y_{补}$同号	上商1	111010	00001
		$2r_3$ /←	110100	0001$\underline{0}$
		-B	+ 001010	
			111110 r_4	

+1　置1

商校正：　左移了4位　$10011\underline{1}_{补}$

余数：　$2^{-4} \times r_4$　数符+1、末尾恒置1

2.4.4 IEEE754浮点四则运算

浮点运算的实现：

低档微机，通过子程序

中档微机，通过浮点处理器（协处理器）

高档微机，通过专门的浮点运算部件

[以IEEE754浮点数为例]

浮点数运算→尾数运算、阶码运算

↓ ↓

原码 移码

↓ ↓

加减乘除 加减

0、IEEE754短浮点数阶码的加减

阶码用非标准移码表示（仅偏移$2^{n-1}-1=127$）

①Ex和Ey分别是两数阶码，Eb是结果阶码，则：

$Eb = Ex+Ey+129 \ （\ \mathrm{mod}\ 2^8）$

原理推导：

$$[Ex+Ey]_{移} = 127 + (Ex+ Ey) = (127+Ex)+(127+Ey) -127$$
$$= [Ex]_{移}+[Ey]_{移}-127$$
$$= [Ex]_{移}+[Ey]_{移}+127_{求补} =[Ex]_{移}+[Ey]_{移}+(-127)_{补}$$
$$= [Ex]_{移} + [Ey]_{移} +10000001$$
$$= [Ex]_{移} + [Ey]_{移} +129 \ （\ \mathrm{mod}\ 2^8）$$

②Ex和Ey分别是两数阶码，Eb是结果阶码，则：

$Eb = Ex - Ey = Ex + [-Ey]_{补} + 127 \quad (\bmod 2^8)$

原理推导：

$[Ex - Ey]_{移} = 127 + (Ex - Ey)$

$= 127 + Ex - Ey - 127 + 127$

$= [Ex]_{移} - [Ey]_{移} + 127$

$= [Ex]_{移} + ([Ey]_{移})_{求补} + 01111111 \quad (\bmod 2^8)$

或者 $= [Ex]_{移} + (-[Ey]_{移})_{补} + 01111111 \quad (\bmod 2^8)$

$\bmod 2^8$，等价于忽略最高位的进位。

[例]若两个阶码分别为10和-5，求10+(-5)和10-(-5)

Ex = 127+10 =137=1000 1001

Ey = 127+ (–5) = 122 = 0111 1010

$[-Ey]_{补}$= 1000 0110

Eb = Ex+Ey +129

= 1000 1001 + 0111 1010 +1000 0001

= 1000 0100 = 132 （mod 2^8）

= +5 正确

```
  1000 1001
  0111 1010
+ 1000 0001
-----------
1 1000 0100
```

Eb = Ex+$[Ey]_{求补}$ +127

= 1000 1001 + 1000 0110 +0111 1111

= 1000 1110 = 142（mod 2^8）

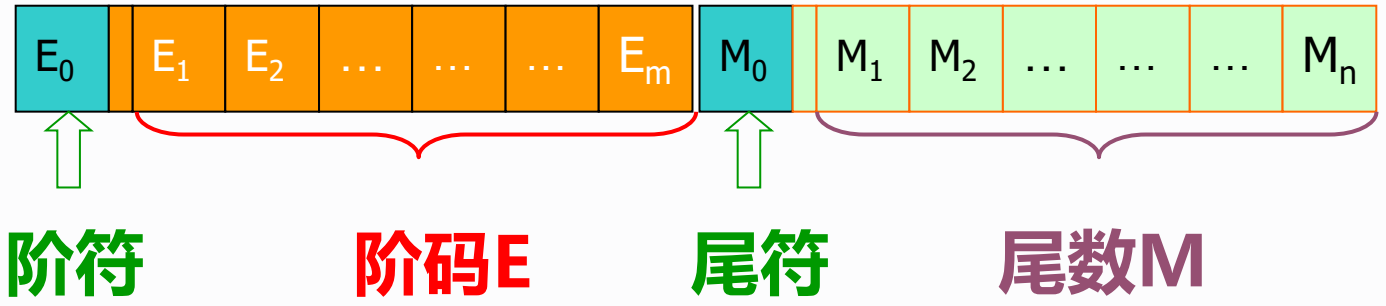
= +15 正确

```
  1000 1001
  1000 0110
+ 0111 1111
-----------
1 1000 1110
```

浮点表示法---知识点回顾

- 对于任意一个二进制数N都可以表示如下：

$$N=2^{E}\times M$$



浮点表示法

- 尾数的表示

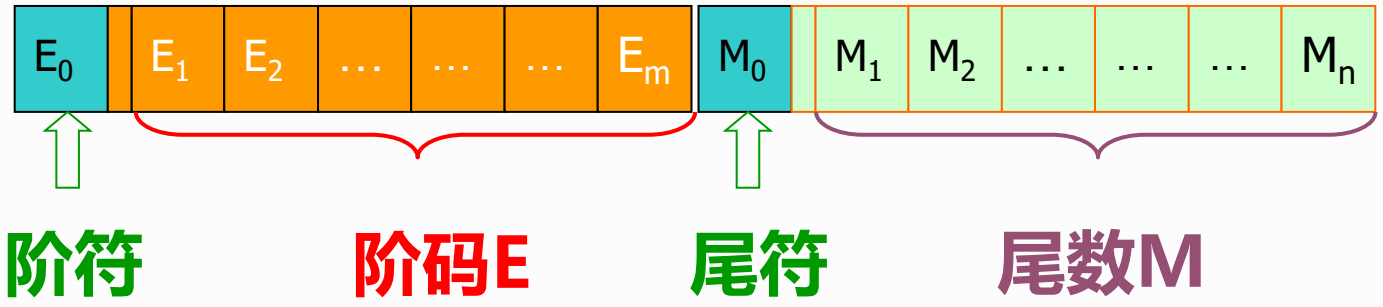
$|M|=0.1\times\times\ldots\times$

$(0.1000001)_{补} = 0.1000001$

$(-0.1000001)_{补} = 1.0111111)_{补}$

对于原码表示的尾数，规格化的特征是：$M_1=1$；

对于补码表示的尾数，规格化的特征是：尾数最高数位和符号位不同。



浮点数加减法五步骤

- 对阶

--阶码

- 尾数运算
- 规格化

--规格化规则

舍入

- 溢出判断

$$X = 2^{E_x} M_x$$
$$+ \quad Y = 2^{E_y} M_y$$
$$????????$$

注： 若参与运算的浮点数阶码和尾数用补码表示，则采用双符号位，便于溢出判断和尾数规格化处理

➢1 “对阶”操作

- 对阶　(使得尾数可以直接相加)
- 例子2^8*(0.11000) + 2^6*(0.00111)

　　大对小　or　小对大　？

　　大阶对小阶 2^8*(0.11000) → 2^6*(11.000)　　溢出

　　小阶对大阶 2^6*(0.00111) → 2^8*(0.0000111)　　？

- **小阶对大阶**　尾数右移

浮点加减运算

“对阶”操作——使两数阶码相等：即比较两个浮点数的阶码值的大小，使两个数的小数点位置对齐。

- **原则：小阶向大阶看齐，**阶码加△E= Ex-Ey ， 尾数右移△E位。即阶小的那个数的尾数右移，右移位数等于两个阶码差的绝对值

浮点加减运算

➢ 2 尾数加减

- 对阶后，两个浮点数的阶码相等，此时可以进行对阶后的尾数相加减。
- 进行加减运算后的尾数不一定是规格化的，因此，浮点数的加减运算需进一步进行规格化处理

➢ 3 运算结果规格化

具体分为以下两种情况：

- a）左规

左规的条件：计算结果的绝对值小于0.5

左规的方法：尾数左移k位直至其绝对值大于0.5，同时阶码减k。

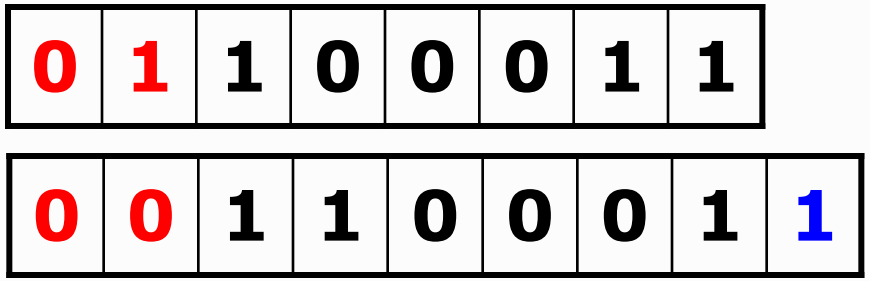
- b）右规

右规的条件：计算结果发生溢出。

右规的方法：尾数右移1位，同时阶码加1。

说明：当尾数计算的结果发生溢出时，并不代表原有浮点数的计算发生溢出，可以通过修改阶码的大小（小数点的位置）使结果正确。

➢ 4 舍入处理



- 右规后低位部分丢失了一位，产生误差
- 舍入方法：**0舍1入，恒舍法，末尾恒置1法。**

➢ 5 溢出处理

1 尾数上溢　右归

　尾数下溢　左归

2 阶码正上溢　浮点数上溢　$|X|\rightarrow\infty$

　阶码负上溢　浮点数下溢　$|X|\rightarrow 0$

3 浮点数溢出判定标准

　阶码运算是否溢出

　溢出检测方法同定点数（3种方法）

溢出判断

以下情况下，可能会导致阶码溢出

- 左规（阶码 - 1）时
  - 左规时：先判断阶码是否为全0，若是，则直接置阶码下溢；否则，阶码减1后判断阶码是否为全0，若是，则阶码下溢。
- 右规（阶码 +1）时
  - 右规（+ 1）时，先判断阶码是否为全1，若是，则直接置阶码上溢；否则，阶码加1后判断阶码是否为全1，若是，则阶码上溢。

浮点运算举例

- 例1　两浮点数 $x = 2^{101} \times 0.11011011$，$y = 2^{111} \times (-0.10101100)$。假设尾数在计算机中以补码表示，可存储10位尾数，2位符号位，阶码以补码表示，双符号位，求 x + y。

解：将x，y转换成浮点格式

$[x]_{浮}$ = 00101，00.11011011

$[Y]_{浮}$ = 00111，11.01010100

步骤1：对阶，阶差为 $E_x - E_y = [E_x]_{补} + [-E_y]_{补}$

$[-E_y]_{补}$=11001　$E_x - E_y = 00101 + 11001 = 11110 = -2 < 0$

小阶对大阶，X阶码加2，尾数右移2位

$[x]_{浮}$ = 00111,00.0011011011　**保留位**

浮点运算举例

$[x]_{浮}$ = 00111, 00.0011011011　　**保留位**

$[Y]_{浮}$ = 00111, 11.01010100

步骤2：尾数求和

$[X+Y]_{浮}$ = 00111, 11.1000101011　　保留位参与运算

步骤3：结果规格化

$[X+Y]_{浮}$ = 00110, 11.000101011　　非规数，左归1位，阶码减1，保留位?

步骤4：舍入处理

$[X+Y]_{浮}$ = 00110, 11.00010110　　(0舍1如法)

$[X+Y]_{浮}$ = 00110, 11.00010101　　(恒置1法)

步骤5：溢出判断

$[X+Y]_{浮} = 2^{110} \times (-0.11101011)$　　**无溢出**

练习

$X=2^{111} \times 0.11111111$，$Y=2^{111} \times 0.10000001$，单符号位

练习-特殊例子

$X=2^{111} \times 0.11111111$，$Y=2^{111} \times 0.10000001$

$$[X]_{浮} = 0111, 0.1111\ 1111$$

$$+\quad [Y]_{浮} = 0111, 0.1000\ 0001$$

$$[X+Y]_{浮} = 0111, 1.1000\ 0000$$

尾数上溢，右归一位，连同符号位进位位一起右移1位，阶码加1

$$[X+Y]_{浮} = 1000, 0.1100\ 0000$$

阶码正溢出，**运算结果上溢**

练习-特殊例子

$X=2^{-1000} \times -0.11110000$，$Y=2^{-1000} \times 0.10000001$

$$[X]_{浮} = 1000, 1.0001\ 0000$$

$$+\quad [Y]_{浮} = 1000, 0.1000\ 0001$$

$$[X+Y]_{浮} = 1000, 1.1001\ 0001$$

尾数下溢，左归一位， 左移一位，阶码减1

$$[X+Y]_{浮} = 1000, 1.0010\ 0010$$

1000+1111=0111 =7

阶码负上溢，**运算结果下溢**

2、浮点数的乘法

设：$A=2^{AE}\times A_M$，$B=2^{BE}\times B_M$

则：$A\times B=(A_M\times B_M)\times 2^{AE+BE}$

浮点乘法➔分解成：移码加法、原码乘法；

※运算步骤：

①求阶和AE+BE

②尾数相乘，

③结果规格化。不需左规！最多右规1次！

④其它处理：

如舍入、置0、阶码溢出判断

3、浮点数的除法

设：$A=2^{AE}\times A_M$，$B=2^{BE}\times B_M$

则：$A\div B=(A_M/B_M)\times 2^{AE-BE}$

浮点除法→分解成：移码减法、原码除法

※运算步骤：

①求阶差AE-BE

②尾数相除，

③结果规格化。

④其它处理：

如舍入、置0、阶码溢出判断